

BitBuild — Project Blueprint & Master Prompt

Interactive Real-Time Game Engine Learning Sandbox & Production AWS Cloud Architecture

TARGET PLATFORM	ARCHITECTURE	ENGINE PERFORMANCE	AI ASSISTANT TARGET
Amazon Web Services (AWS)	Serverless / Edge SPA	60 FPS Canvas Loop	ChatGPT / Codex / Claude

1. Executive Summary & Core Product Concept

BitBuild is an interactive educational web platform designed to teach game development, physics simulation, and autonomous artificial intelligence by empowering students to write and modify real code running inside a live 60 FPS HTML5 Canvas game engine.

Instead of viewing abstract tutorials, every single lesson is an executable simulation. When users modify code in the embedded editor and hit **Run ▶**, their code directly drives the game loop physics, vector calculations, collision responses, and AI steering behavior.

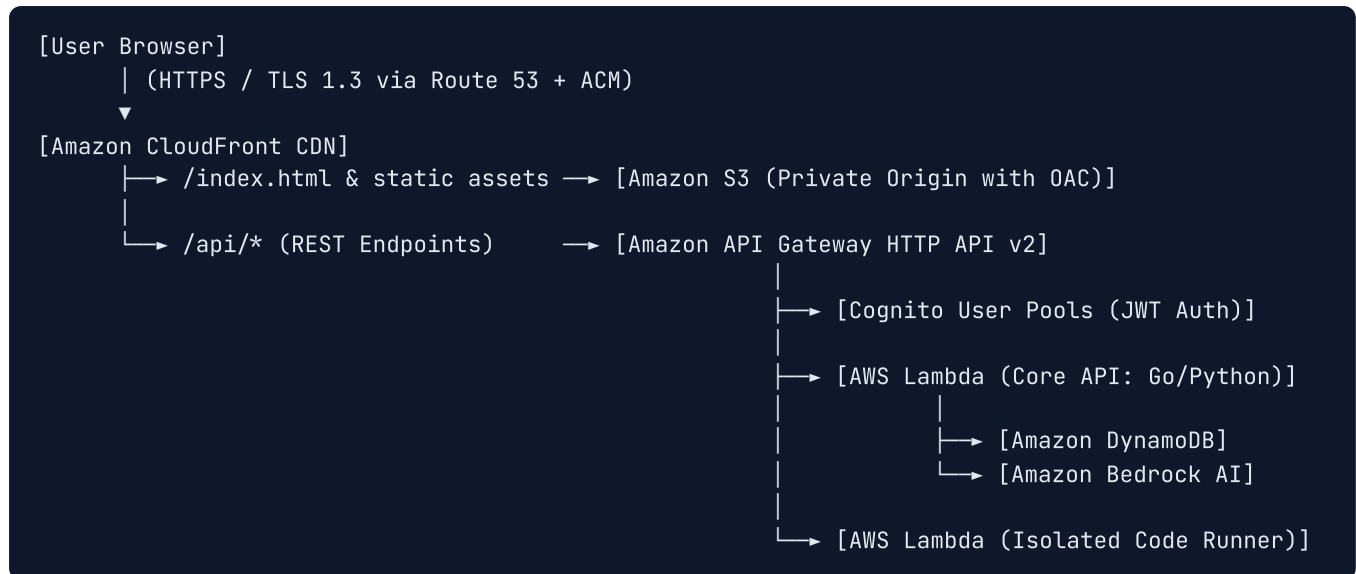
Key Innovation: Client-side execution delivers instantaneous feedback with zero network latency, while AWS cloud services manage identity, persistent progression, lesson draft synchronization, and AI-powered bug diagnostics.

2. Recommended Technology Stack

Layer	Recommended Stack	Role & Architectural Rationale
Frontend UI	React 19 / Next.js + Vanilla CSS / Tailwind	Ultra-responsive client interface; custom dark terminal design system.
In-Browser Editor	CodeMirror 6	Modular, high-performance editor with syntax highlighting for JS, Python, and C++.
Game Engine	HTML5 2D Canvas + Particle Emitter	Deterministic 60 FPS frame-loop with delta-time (<code>dt</code>) integration.
Audio Synthesizer	Native HTML5 Web Audio API	Procedural 8-bit oscillator voice blips (Undertale/Celeste style) with zero asset overhead.
Backend API	Go (Golang) / Python on AWS Lambda	Sub-15ms cold start serverless API handling user state, submissions, and telemetry.
Database	Amazon DynamoDB (On-Demand)	Sub-10ms key-value persistence for student XP, level completions, and code drafts.
Edge Hosting	Amazon CloudFront + S3	Global CDN edge caching, sub-50ms TTFB worldwide, 99.999% SLA, near-\$0 idle cost.
AI Diagnostics	Amazon Bedrock (Claude 3.5 Sonnet)	Understands student code mistakes, syntax exceptions, and dt frame-rate coupling.

3. AWS Cloud Hosting Architecture

BitBuild is designed to deploy on **Amazon Web Services (AWS)** using a cloud-native, serverless, cost-effective architecture:



Key Architectural Benefits:

- **Scales to Zero:** In periods of zero traffic, monthly AWS cost is virtually \$0 (pay-per-request).
- **Global Acceleration:** CloudFront edge caches static JS/CSS and WASM bundles near users across 600+ Points of Presence.
- **Zero Server Maintenance:** No EC2 instances to patch, scale, or configure. Everything runs as managed serverless infrastructure.

4. Simulation Engine Contract & Runtime Interface

The simulation engine executes user-written code on every frame inside the `requestAnimationFrame` loop via the evaluated function contract:

```

type UpdateFn = (
  state: PlayerState,
  keys: { left: boolean; right: boolean; up: boolean; down: boolean },
  dt: number,          // Delta time in seconds (~0.016s at 60 FPS)
  world: WorldConfig
) => void;

interface PlayerState {
  x: number;           // Player X coordinate
  y: number;           // Player Y coordinate
  vx: number;          // Horizontal velocity
  vy: number;          // Vertical velocity
  width: number;       // Sprite width (22px)
  height: number;      // Sprite height (22px)
  onGround: boolean;   // Ground collision flag
  enemyX: number;      // Chasing AI enemy X coordinate
  enemyY: number;      // Chasing AI enemy Y coordinate
  score: number;       // Points collected
  coins: { x: number; y: number; collected: boolean }[];
}

interface WorldConfig {
  width: number;       // 480 px
  height: number;      // 280 px
  groundY: number;     // 240 px
  platform: { x: number; y: number; w: number; h: number };
  goal: { x: number; y: number; w: number; h: number };
}

```

5. Curriculum Roadmap & Code Implementations

BitBuild includes a progressive, 4-chapter curriculum guiding students from basic velocity to advanced game mechanics:

Lesson 01 — Movement & Delta-Time Scaling (50 XP)

Core Concept: Input handling, frame-rate independence via `dt`, coordinate bounds clamping.

```
// JavaScript Implementation
const speed = 150; // pixels per second

if (keys.left) state.x -= speed * dt;
if (keys.right) state.x += speed * dt;
if (keys.up) state.y -= speed * dt;
if (keys.down) state.y += speed * dt;

// Keep player inside canvas boundaries
state.x = Math.max(0, Math.min(world.width - state.width, state.x));
state.y = Math.max(0, Math.min(world.height - state.height, state.y));
```

Lesson 02 — Gravity, Jumping & Ground State (75 XP)

Core Concept: Gravitational acceleration (`vy += gravity * dt`), jump impulse, landing collision.

```
// JavaScript Implementation
const gravity = 800;
const jumpPower = 380;
const moveSpeed = 160;

if (keys.left) state.vx = -moveSpeed;
else if (keys.right) state.vx = moveSpeed;
else state.vx = 0;

state.vy += gravity * dt;

if (keys.up && state.onGround) {
  state.vy = -jumpPower;
  state.onGround = false;
}

state.x += state.vx * dt;
state.y += state.vy * dt;

const groundY = world.groundY - state.height;
if (state.y ≥ groundY) {
  state.y = groundY;
  state.vy = 0;
  state.onGround = true;
} else {
  state.onGround = false;
}

state.x = Math.max(0, Math.min(world.width - state.width, state.x));
```

Lesson 03 — AABB Spatial Collision Detection (100 XP)

Core Concept: 2D Axis-Aligned Bounding Box overlap tests and platform landing collision response.

```
// JavaScript Implementation
const p = world.platform;
let nextX = state.x + state.vx * dt;
let nextY = state.y + state.vy * dt;

const hitsPlatform =
  nextX < p.x + p.w && nextX + state.width > p.x &&
  nextY < p.y + p.h && nextY + state.height > p.y;

if (!hitsPlatform) {
  state.x = nextX;
  state.y = nextY;
} else {
  if (state.vy > 0 && state.y + state.height ≤ p.y + 10) {
    state.y = p.y - state.height;
    state.vy = 0;
    state.onGround = true;
  } else {
    state.vy = 0;
  }
}
```

Lesson 04 — Autonomous Steering & Enemy AI (120 XP)

Core Concept: Vector subtraction `(dx, dy)`, distance normalization, proportional pursuit velocity.

```
// JavaScript Implementation
const enemySpeed = 95;

const dx = state.x - state.enemyX;
const dy = state.y - state.enemyY;
const dist = Math.sqrt(dx * dx + dy * dy) || 1;

state.enemyX += (dx / dist) * enemySpeed * dt;
state.enemyY += (dy / dist) * enemySpeed * dt;
```

Lesson 05 — Boss Capstone: Arena Escape (250 XP)

Core Concept: Integrated physics platformer combining jumping, platform collision, dodging chasing AI, collecting 3 energy gems, and reaching the green exit portal.

Celebration FX: When win condition is satisfied, the platform triggers full-screen `canvas-confetti` and synthesizes a 4-note ascending C-major victory arpeggio via Web Audio API.

6. Audio Synthesizer Engine & AI Mentor

- **Procedural Voice Blips:** Employs Web Audio API oscillators (triangle waves 440-620Hz for Mentor Pixel, square waves 220-340Hz for Mentor Byte) synchronized with typewriter character text generation.
- **"Explain My Bug" Diagnostics:** Context-aware diagnostic modal that detects frame-rate coupling (missing `dt` multiplication), variable reference errors, and collision tunneling with instant reference code fixes.

7. Project Directory Layout

```
bitbuild/
├── infra/                                # AWS CDK / Terraform Infrastructure
│   ├── lib/frontend-stack.ts           # CloudFront + S3 + ACM + Route 53
│   ├── lib/api-stack.ts                # API Gateway + Lambda + DynamoDB
│   └── cdk.json
├── backend/                             # Serverless Go / Python Handlers
│   ├── cmd/api/main.go                 # API Gateway handler endpoint
│   ├── internal/handlers/              # User progress, AI diagnostics endpoints
│   └── go.mod
├── frontend/                            # Client Application (React 19 / Vite)
│   ├── src/
│   │   ├── components/Canvas/         # 60 FPS HTML5 Canvas engine & renderer
│   │   ├── components/Editor/         # CodeMirror 6 with Run and Reset
│   │   ├── components/Mentor/         # Typewriter dialogue & audio blips
│   │   ├── services/BlipAudio.ts      # Web Audio API synthesizer
│   │   ├── services/PythonRunner.ts  # Client-side transpiler & evaluator
│   │   ├── data/lessons.ts            # Curriculum data & starter code
│   │   └── App.tsx                    # Core orchestrator
│   └── package.json
└── README.md
```

8. Master Prompt for AI Coding Assistants

When giving this specification to ChatGPT or Codex, use the prompt below:

Prompt to feed to ChatGPT:

"You are a Principal Software Architect and Full-Stack Game Engine Engineer. Above is the complete project specification for BitBuild, an interactive 60 FPS game programming learning platform hosted on AWS. Review the architecture, data contracts, and simulation interface, and help me implement / extend features according to this standard."